

Solution of Mid Term Examination, Autumn 2025

Course Title: Compiler, Course Code: CSE-3527

1. a)

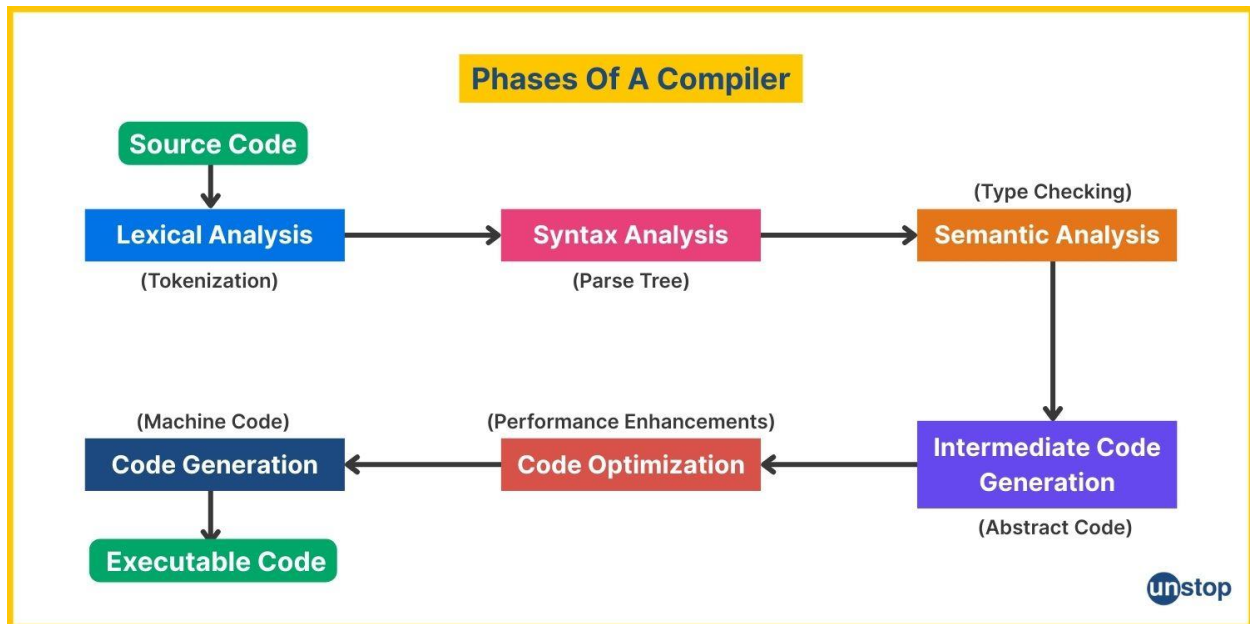
(2)

Define a program with a proper block diagram that is used to compile a program in high-level languages and contrast it with the interpreter.

Compiler

A **compiler** is a program that translates a **high-level language program into machine code all at once before execution.**

Block Diagram of Compiler



Compiler Phases

1. **Lexical Analysis**
 - Converts characters into tokens.
2. **Syntax Analysis**
 - Checks grammatical structure using CFG.
3. **Semantic Analysis**
 - Checks meaning, type compatibility.
4. **Intermediate Code Generation**
 - Produces intermediate representation.
5. **Code Optimization**
 - Improves efficiency.

6. Code Generation

- Produces machine code.

Interpreter

An **interpreter** translates and executes the program **line by line** without producing machine code separately.

Difference Between Compiler and Interpreter

Feature	Compiler	Interpreter
Translation	Entire program	Line by line
Speed	Faster execution	Slower
Error detection	After compilation	Immediate
Example	C, C++	Python, JavaScript

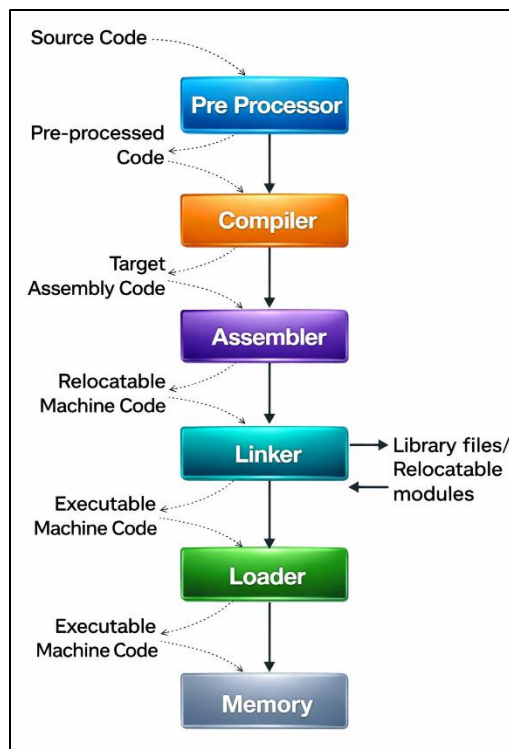
Or

(2)

Explain the working of a language processing system that converts a high-level source program into an executable target code. Illustrate the process with an appropriate system diagram.

A language processing system converts a **high-level source program** into **executable machine code**.

System Diagram



Steps

1. **Source Program**
 - Written in high-level language.
2. **Compiler**
 - Converts source code → assembly code.
3. **Assembler**
 - Converts assembly code → machine code.
4. **Linker**
 - Links library functions.
5. **Loader**
 - Loads program into memory for execution.

b)

(4+1+3)

Consider the following C program:

```
int main() {  
    float salary, basic, increment, RATE;  
    scanf("%f%f%f", basic, increment, RATE);  
    salary = basic + increment * RATE;  
    printf("%f\n", salary);  
    return 0;  
}
```

Translate this C program step by step through the compiler phases. Draw the diagram of the compiler's phases with output translating at each phase and show the appropriate symbol table entries for identifiers, syntax definition in context-free grammar (CFG), and tokenization for each lexeme.

[Assume all variables are floating type]

1. Lexical Analysis (Tokenization)

Lexeme	Token
int	keyword
main	identifier
float	keyword
salary	identifier
basic	identifier
increment	identifier
RATE	identifier
scanf	identifier
printf	identifier
=	assignment
+	operator
*	operator

return	keyword
0	constant

2. Symbol Table

Identifier	Type	Attribute
salary	float	variable
basic	float	variable
increment	float	variable
RATE	float	variable
main	function	identifier

3. Syntax Definition (CFG)

program $\rightarrow$ declaration statement

declaration $\rightarrow$ type identifier

type $\rightarrow$ int | float

statement $\rightarrow$ assignment

assignment $\rightarrow$ identifier = expression

expression $\rightarrow$ expression + term

term $\rightarrow$ term * factor

factor $\rightarrow$ identifier

4. Intermediate Code

Three-address code:

```
t1 = increment * RATE
t2 = basic + t1
salary = t2
```

5. Code Optimization

```
t1 = increment * RATE
salary = basic + t1
```

6. Target Code

Example assembly:

```
LOAD increment
MUL RATE
ADD basic
STORE salary
```

2. a)

(1+1+2)

What do you mean by syntax definition in compiler construction? Describe each of the basic syntax definition components of a language in compiler design. Give syntax definition for the following conditional if-else statements in C (programming):

```
if (conditions) {
    statements;
}
else if (conditions) {
    statements;
}
...
else if (conditions) {
    statements;
}
else {
    statements;
}
```

Syntax definition specifies the **structure of valid statements in a programming language using grammar rules (CFG).**

Basic Components

1. Terminal Symbols

Actual tokens.

Example:

```
if , else , identifier
```

2. Non-Terminal Symbols

Variables representing structures.

Example:

statement
expression
condition

3. Production Rules

Define syntax.

Example:

statement $\rightarrow$ if (condition) statement

Syntax Definition of Conditional If-Else

$S \rightarrow \text{if } (C) \ S$
 $S \rightarrow \text{if } (C) \ S \text{ else } S$
 $S \rightarrow \text{statement}$
 $C \rightarrow \text{condition}$

Extended form:

$S \rightarrow \text{if}(C) \ S \ E$
 $E \rightarrow \text{else if}(C) \ S \ E$
 $E \rightarrow \text{else } S$

Or

Describe ambiguity in CFG and its impacts on parser design? Consider the following CFG **G**:

$\text{digit} \rightarrow \text{digit} + \text{digit} \mid \text{digit} - \text{digit}$
 $\text{digit} \rightarrow \text{digit} * \text{digit} \mid \text{digit} / \text{digit}$
 $\text{digit} \rightarrow 0 \mid \dots \mid 9$

Whether the above grammar is ambiguous or not. Justify your answer using derivation or parse tree for the input string “5*2+3-5”.

If it is ambiguous, convert it into an unambiguous grammar using appropriate variables.

Construct the syntax tree and left-most derivation for the input string “5*2+3-5”.

A CFG is ambiguous if a string can have more than one parse tree or derivation.

Given CFG

$\text{digit} \rightarrow \text{digit} + \text{digit} \mid \text{digit} - \text{digit}$
 $\text{digit} \rightarrow \text{digit} * \text{digit} \mid \text{digit} / \text{digit}$
 $\text{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Input String

$5 * 2 + 3 - 5$

This grammar is **ambiguous** because:

$(5 * 2) + (3 - 5)$

or

$5 * (2 + 3) - 5$

Both parses possible.

Unambiguous Grammar

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow \text{digit}$

$\text{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Leftmost Derivation

E

E - T

E + T - T

T + T - T

T * F + T - T

F * F + T - T

5 * 2 + 3 - 5

b)

(2+2)

Define a CFG for the language of **0 and 1** to generate the palindrome string.

Explain the roles of the **lexical analyzer** and **parser** in a compiler with the appropriate block diagram.

A **palindrome** reads the same forward and backward.

CFG

$S \rightarrow 0S0$

$S \rightarrow 1S1$

$S \rightarrow 0$

$S \rightarrow 1$

$S \rightarrow \varepsilon$

Examples:

```
0
1
101
0110
10001
```

Roles of Lexical Analyzer

Functions:

- Reads input characters
- Groups them into tokens
- Removes spaces
- Updates symbol table

Example:

```
salary = basic + rate
```

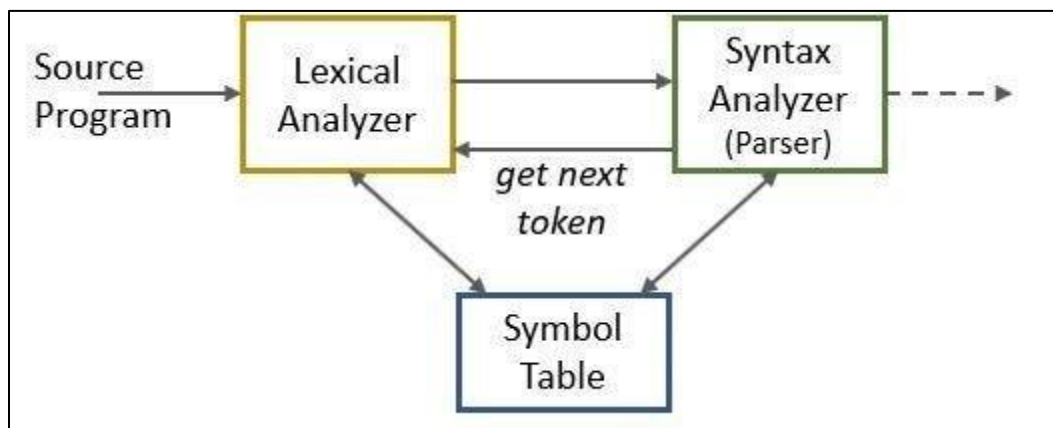
Tokens:

```
identifier = identifier + identifier
```

Role of Parser

- Receives tokens from lexical analyzer
- Checks grammar rules
- Builds parse tree

Block Diagram



c)

(2)

Define all four basic types of grammar by emphasizing the production rules of the grammar with appropriate examples.

Four Basic Types of Grammar (Chomsky Hierarchy)

Grammars are classified into **four types based on their production rules.**

Type	Name	Production Rule
Type-1	Unrestricted Grammar	$\alpha \rightarrow \beta$
Type-2	Context Sensitive Grammar	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type-3	Context Free Grammar	$A \rightarrow \gamma$
Type-4	Regular Grammar	$A \rightarrow aB$ or $A \rightarrow a$

Examples

Regular Grammar

$S \rightarrow aA$

$A \rightarrow bB$

$B \rightarrow c$

Context Free Grammar

$S \rightarrow aSb$

$S \rightarrow \varepsilon$

Context Sensitive Grammar

$AB \rightarrow ABC$

$BC \rightarrow bC$

Unrestricted Grammar

$AB \rightarrow BA$

$A \rightarrow a$

$B \rightarrow b$

3. a)

(1+1+1+1.5+1.5)

Write the **regular expression** and draw the **nondeterministic finite automata (NFA)**, **deterministic finite automata (DFA)** transition diagram, **ϵ -NFA** and **CFG** to define an identifier for C programming language $\Sigma = \{a-z, A-Z, _, 0-9\}$ transition diagram to define the number of the C programming language.

Identifier in C Programming Language

An **identifier** in C programming language is a name used to identify variables, functions, arrays, etc.

Rules of Identifier

1. The first character must be a **letter (a–z, A–Z) or underscore (_)**.
2. The remaining characters may be **letters, digits (0–9), or underscore (_)**.

Alphabet:

$$\Sigma = \{a-z, A-Z, _, 0-9\}$$

Examples of valid identifiers:

```
sum
_rate
value1
num_2
```

Examples of invalid identifiers:

```
1sum
9value
```

Regular Expression

The regular expression for a C identifier is:

$$[A-Za-z_][A-Za-z0-9_]*$$

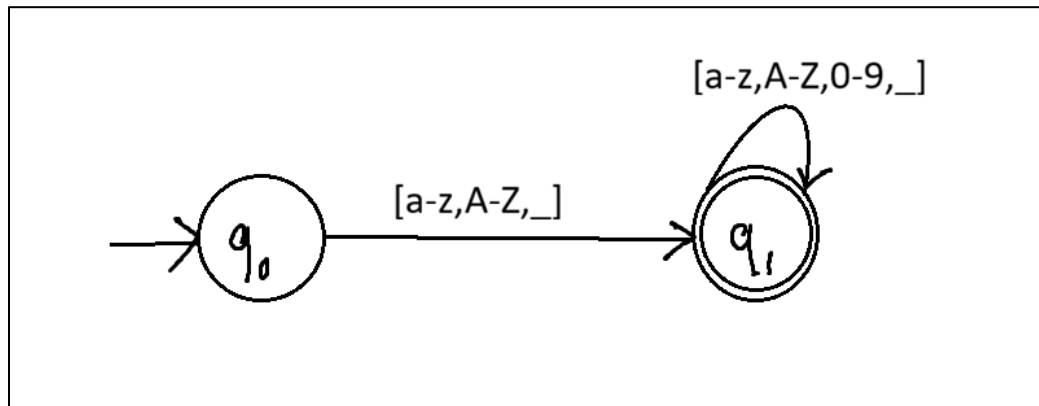
Explanation:

- $[A-Za-z_]$ $\rightarrow$ first character must be a letter or underscore
- $[A-Za-z0-9_]*$ $\rightarrow$ remaining characters can be letter, digit, or underscore

Nondeterministic Finite Automata (NFA)

The NFA accepts identifiers where the first character is a **letter or underscore**, and the remaining characters may be **letters, digits, or underscores**.

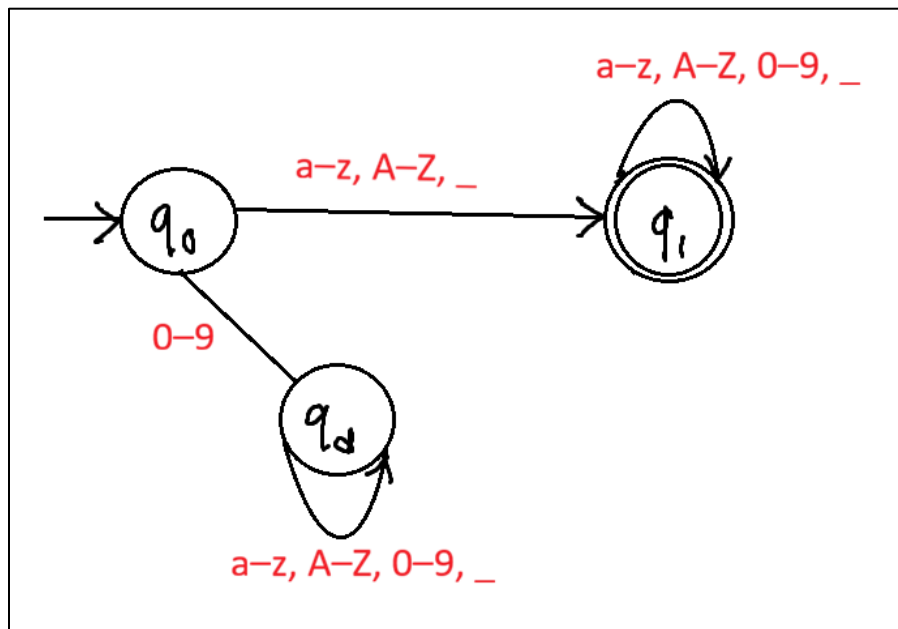
State Diagram



Deterministic Finite Automata (DFA)

A DFA for identifiers ensures the first character is **not a digit** and moves to a **dead state** if it occurs.

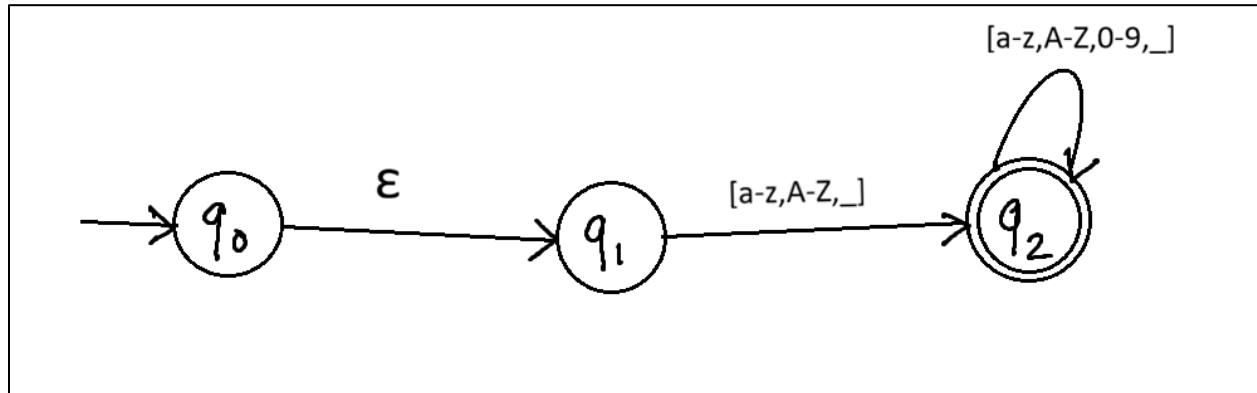
State Diagram



ϵ -NFA (Epsilon NFA)

An ϵ -NFA allows an **epsilon transition** before reading the first input symbol.

State Diagram



Context Free Grammar (CFG)

A context-free grammar for identifiers can be defined as:

$ID \rightarrow \text{letter } ID'$

$ID' \rightarrow \text{letter } ID' \mid \text{digit } ID' \mid _ ID' \mid \epsilon$

$\text{letter} \rightarrow a|b|c|\dots|z|A|B|C|\dots|Z|_$

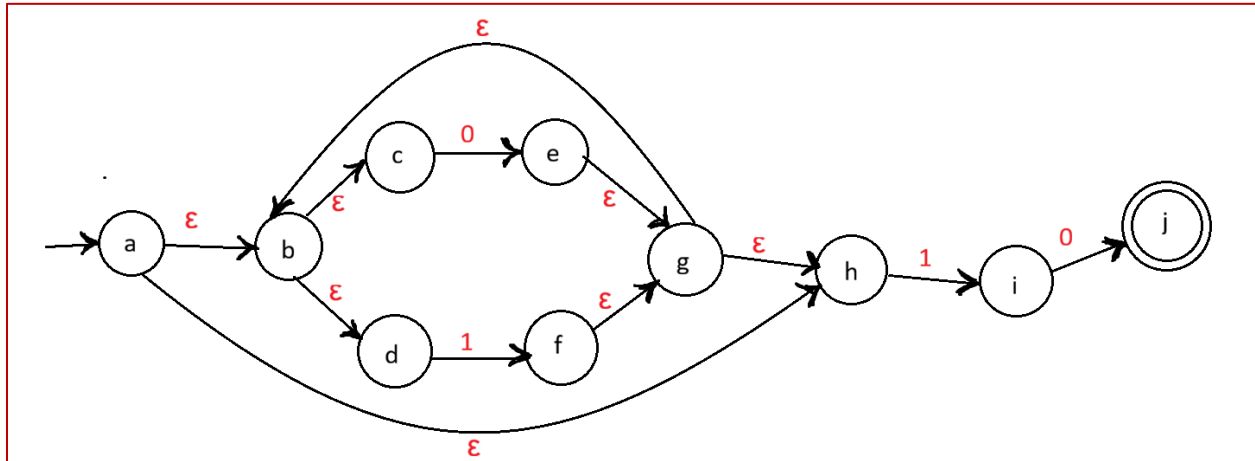
$\text{digit} \rightarrow 0|1|2|3|4|5|6|7|8|9$

This grammar generates all valid identifiers in the C programming language.

b)

(2+2)

Consider the following ϵ -NFA to define the all language $\Sigma = \{0,1\}$ end with **one and zero**. Convert it into the **DFA** using the ϵ -closure technique **step by step**:



Language Definition

Consider the ϵ -NFA over the alphabet

$$\Sigma = \{0, 1\}$$

that recognizes all binary strings ending with **10** (one followed by zero).

So the language can be written as:

$$L = \{ w \in \{0,1\}^* \mid w \text{ ends with } 10 \}$$

This means any sequence of **0s and 1s** is allowed, but the **last two symbols must be 1 then 0**.

Valid strings (examples):

$$L = \{10, 110, 0010, 1010, 00010, 01010, \dots\}$$

ϵ -NFA Transition Table

State	0	1	ϵ
$\rightarrow a$	–	–	b, h
b	–	–	c, d
c	e	–	–
d	–	f	–

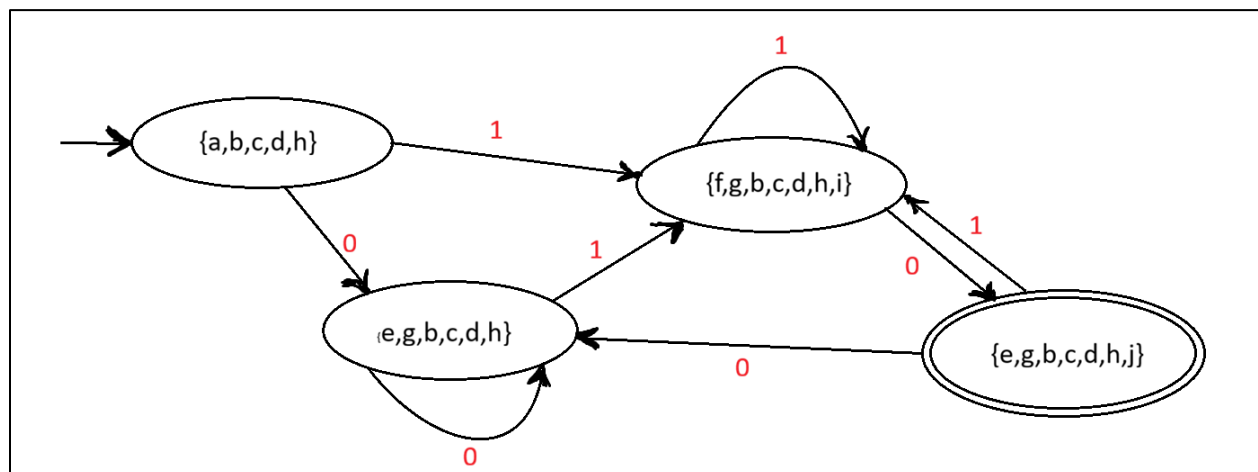
e	-	-	g
f	-	-	g
g	-	-	b, h
h	-	i	-
i	j	-	-
*j	-	-	-

ϵ -Closure Table

State	ϵ -closure
a	{a, b, c, d, h}
b	{b, c, d}
c	{c}
d	{d}
e	{e, g, b, c, d, h}
f	{f, g, b, c, d, h}
g	{g, b, c, d, h}
h	{h}
i	{i}
j	{j}

DFA Table

DFA State	0	1
$\rightarrow\{a,b,c,d,h\}$	{e,g,b,c,d,h}	{f,g,b,c,d,h,i}
{e,g,b,c,d,h}	{e,g,b,c,d,h}	{f,g,b,c,d,h,i}
{f,g,b,c,d,h,i}	{e,g,b,c,d,h,j}	{f,g,b,c,d,h,i}
*{e,g,b,c,d,h,j}	{e,g,b,c,d,h}	{f,g,b,c,d,h,i}



Or

(1+3)

Differentiate between **DFA and NFA**.

Describe the language $\Sigma = \{a, b\}$ in ϵ -NFA for the regular expression: **a (a | b) * a**

Difference between DFA and NFA

Feature	DFA	NFA
Transition	One per symbol	Multiple possible
ϵ transitions	Not allowed	Allowed
Deterministic	Yes	No
Complexity	Easy implementation	Theoretical model

ϵ -NFA for Regular Expression

Regular Expression:

$(a | b)^* a$

Meaning:

- Any combination of **a and b**
- Ending with **a**

Examples:

a
ba
aba
abba
bbba

